

Projet final FastAPI

Le projet final est réalisé par groupe de 3 à 5 étudiants (3 fortement conseillé).

L'objectif est de concevoir une API cohérente répondant au besoin métier de votre choix. Le projet doit mettre en pratique l'ensemble des notions abordées tout au long du module.

1. Volume du projet

Dans un souci d'équité entre les groupes, la charge de travail varie en fonction du nombre d'étudiants.

Taille du groupe	Ressources CRUD minimum	Relations métier minimum	Fonctionnalités métier avancées minimum
3 étudiants	6	3	2
4 étudiants	8	4	2
5 étudiants	10	5	3

Ressources CRUD

Chaque étudiant doit être responsable d'au moins 2 ressources CRUD complètes.

Une ressource CRUD doit correspondre à une véritable entité métier et proposer les opérations suivantes :

- Création
- Lecture (GetAll & GetOne)
- Modification (PUT ou PATCH au choix selon le contexte fonctionnel)
- Suppression

Exemples :

- API de gestion de recettes de cuisine : recette, ingrédient, utilisateur, catégorie, commentaire, note, liste de courses, menu, allergène, ustensile...

- API de gestion d'un festival : festival, artiste, concert, scène, utilisateur, billet, réservation, stand, partenaire, programmation.
- API de gestion d'une salle de sport : utilisateur, exercice, séance, programme, équipement, coach, abonnement, salle, réservation, objectif sportif.

Relations métier

L'API doit contenir au moins une relation métier significative par étudiant.

Exemples :

- Recettes de cuisine : une recette contient plusieurs ingrédients ; un utilisateur crée plusieurs recettes.
- Festival : un artiste participe à plusieurs concerts ; une scène accueille plusieurs concerts.
- Salle de sport : un programme contient plusieurs exercices ; un coach encadre plusieurs utilisateurs.

Fonctionnalités métier avancées

Le projet doit comporter des fonctionnalités dépassant les opérations CRUD classiques.

Une fonctionnalité métier avancée doit mobiliser plusieurs éléments de l'application : récupération de données, vérification de règles métier, manipulation de plusieurs ressources, interaction avec une API externe, etc.

Exemples :

- Recettes de cuisine : générer une liste de courses depuis plusieurs recettes ; adapter les quantités au nombre de personnes.
- Festival : détecter les conflits de programmation ; générer le planning personnalisé d'un festivalier.
- Salle de sport : générer une séance selon un objectif ; vérifier les disponibilités avant une réservation.

2. Grille de notation du code :

Critère	Points	Attendus
Richesse fonctionnelle et modélisation	4 pts	Respect du nombre minimum de CRUD, relations et fonctionnalités métier selon la taille du groupe. Ressources pertinentes et cohérentes. Relations réellement exploitées dans l'application.
Validation des données avec Pydantic	1,5 pt	Modèles Pydantic pertinents, typage correct, champs obligatoires/optionnels et contraintes de validation. Les entrées incorrectes sont correctement rejetées.
Base de données et persistance	2 pts	Utilisation de PostgreSQL. Modélisation cohérente des entités et relations. Les CRUD et fonctionnalités métier utilisent réellement la base de données.
Authentification JWT	1,5 pt	Authentification fonctionnelle, génération et vérification des JWT, identification correcte de l'utilisateur authentifié.
Sécurisation des routes	1 pt	Au moins deux routes pertinentes sont protégées. Gestion correcte des accès non authentifiés/non autorisés et des codes HTTP associés.
Interaction avec une API tierce	1,5 pt	Consommation d'au moins une API externe. Les données obtenues sont réellement intégrées dans une fonctionnalité du projet. Gestion des principales erreurs du service externe.
Architecture du projet	1,5 pt	Organisation claire et cohérente des dossiers et modules. Architecture adaptée à la taille du projet et facilement maintenable.

Séparation des responsabilités	1,5 pt	Séparation entre routes/controllers, services/logique métier, modèles/schemas et accès aux données. Les routes restent légères.
Tests unitaires et tests API	2 pts	Tests automatisés exécutables couvrant la logique métier et les principaux endpoints. Cas nominaux et cas d'erreur présents.
Gestion des erreurs HTTP	1 pt	Gestion des ressources inexistantes, données incohérentes, accès interdits, conflits, etc. Codes HTTP et <code>HTTPException</code> correctement utilisés.
Respect des conventions REST	1 pt	Utilisation cohérente des verbes HTTP, URLs, path/query parameters et codes de statut.
Documentation et exploitabilité	1 pt	Swagger/OpenAPI exploitable et README permettant l'installation, la configuration et le lancement du projet.
Qualité du code et utilisation de Git	0.5 pt	Code lisible et correctement nommé, duplication limitée, <code>.gitignore</code> , absence de secrets et historique Git permettant d'identifier la participation des différents membres.
Total	20 pts	